Claude Multi-Account Fine Tuning

llmctl Integration — FAQ
- General
  - Do I need to install llmctl through claude_toolkit?
  - Does this work with Kimi Code as well as Claude Code?
  - What if llmctl isn't installed on my host at all?
  - Can I run multiple llmctl models at once through claude_toolkit?
- Detection and naming
  - Why don't I see my llmctl model as an alias?
  - Why is the alias named `llmctl-fast` and not just `fast`?
  - What happens to my other Claude Code accounts/providers when I switch llmctl models?
- The two warnings
  - I got a warning about my model being reachable from my local network — is that a bug?
  - I got a context-size warning / my session failed with a context-exceeded error on an llmctl model — what do I do?
- Troubleshooting
  - I ran `llmctl switch` myself and now claude_toolkit's alias seems confused
  - A switch seems to hang or fail with an unfamiliar message
  - Where do I report a problem that's actually inside llmctl itself, not this integration?
- Validation and Vision
  - Does claude_toolkit validate my local Vision model?
  - What does exit 2 / UNDETERMINED mean?
  - Why does Vision QA (§11.4.278) matter here?
  - Is claude_toolkit part of the constitution?

llmctl Integration — FAQ

See also: Quickstart · User Guide

General

Do I need to install llmctl through claude_toolkit?

No. llmctl is a separate tool you install and manage yourself — build its engines, download its model profiles, start/stop/switch them with its own CLI. claude_toolkit never installs, configures, or manages llmctl; it only *recognizes* whatever profile you already have running and exposes it as an alias.

Does this work with Kimi Code as well as Claude Code?

Yes, both. An llmctl profile named `fast` becomes `llmctl-fast` for Claude Code and `kimi-llmctl-fast` for Kimi Code, pointed at the same running model.

What if llmctl isn't installed on my host at all?

Nothing breaks. With no llmctl binary and no pins file present, detection returns zero records, zero `llmctl-*` aliases exist, and no error is raised — exactly as if llmctl were simply absent.

Can I run multiple llmctl models at once through claude_toolkit?

Detection supports it — if llmctl is running several profiles simultaneously, each gets its own alias (`llmctl-fast`, `llmctl-vision`, etc.) at the same time. But *switching* between them through claude_toolkit's own on-demand mechanism is exclusive: launching a different `llmctl-<profile>` alias always stops whichever profile was previously active and starts exactly the one you asked for, mirroring llmctl's own `switch` command. This was a deliberate design decision (not co-residency) made explicitly for this integration — see the feature's Open Questions / Clarifications for the reasoning.

Detection and naming

Why don't I see my llmctl model as an alias?

Two likely causes. First, claude_toolkit hasn't re-synced since you started the profile — run a sync (`claude-providers sync`, or your usual account-sync command) and it will pick it up. Second, and more subtly: the profile has to be *genuinely answering its own `/v1/models` endpoint* — a port merely being open isn't enough. If

something else is occupying that port, or the model is still loading, the alias won't appear until it actually responds. Check `llmctl status` directly to confirm the profile is really up.

Why is the alias named `llmctl-fast` and not just `fast`?

Always namespaced, on purpose. A bare name like `fast` or `coder` could collide with — or just be confused for — an unrelated alias, and an operator should be able to tell at a glance whether they're about to talk to a local model or a cloud provider. `llmctl-<profile>` is never abbreviated to the bare profile name.

What happens to my other Claude Code accounts/ providers when I switch llmctl models?

Nothing. Switching is entirely scoped to llmctl-backed aliases — launching or losing one has zero effect on any native account alias (`claude1`, etc.) or any other provider alias.

The two warnings

I got a warning about my model being reachable from my local network — is that a bug?

No. By default, llmctl binds its model-serving ports to `0.0.0.0` (every network interface), not just `127.0.0.1` — that's llmctl's own documented trade-off (its OpenAI-compatible API has no built-in authentication, so this is a deliberate operator-facing choice, not an accident). claude_toolkit can't change that default and doesn't try to; it just reads the real listening-socket state and tells you honestly when a profile is reachable beyond localhost, rather than staying silent about it. The alias still works either way. If you want a given profile loopback-only, set llmctl's own `LLMCTL_BIND_HOST=127.0.0.1` (globally) or `LLMCTL_BIND_HOST_<PROFILE>=127.0.0.1` (per profile) before starting it — see the User Guide for details.

I got a context-size warning / my session failed with a context-exceeded error on an llmctl model — what do I do?

This is the single most important thing to know about using llmctl models through a CLI agent. A real, captured example from this integration's own test evidence:

```
error: failed to run prompt: provider.api_error: 400 request
(92629 tokens)
```

```
exceeds the available context size (8192 tokens), try increasing
it
```

That's the `llmctl-small` profile (a small Llama-3.2-3B model) — its real, configured context window is 8192 tokens, but a single Kimi Code turn needed 92,629 tokens just for its own system prompt and tool schemas before your actual conversation even started. This isn't a bug in claude_toolkit, and it isn't something detection can silently fix: it's a property of how that specific llmctl profile is configured. claude_toolkit's job is to surface it honestly — the alias still appears, but with an explicit warning naming the real context and the shortfall — rather than let you discover it mid-session with a confusing provider error. If you hit this, either use a larger-context profile for agentic/CLI work, or keep the small one for simpler, non-agentic use directly against its raw API.

Troubleshooting

I ran `llmctl switch` myself and now claude_toolkit's alias seems confused

You shouldn't normally need to run `llmctl switch` by hand — launching a different `llmctl-<profile>` alias already does this for you automatically. If you do switch manually outside claude_toolkit, the next alias launch (or sync) will simply observe whatever llmctl's own `status` reports as the live profile; there's no separate state for claude_toolkit to get out of sync, since it never assumes a profile is active — it checks.

A switch seems to hang or fail with an unfamiliar message

llmctl's own `switch` command stops every other running profile and starts exactly the one you asked for, restoring the previous set if the new one fails to start. In the rare case where that restoration *also* fails, claude_toolkit surfaces a distinctly-worded, higher-severity warning (prefixed `CRITICAL:`) rather than treating it like an ordinary refusal — if you see that prefix, check `llmctl status` and `llmctl logs <profile>` directly, since it means llmctl itself ended the operation with fewer services running than before.

Where do I report a problem that's actually inside llmctl itself, not this integration?

claude_toolkit doesn't modify llmctl's own codebase — any defect found *inside* llmctl during this integration's development is documented, not patched here. See `../research/2026-10-02-llmctl-upstream-findings.md` for the current record of what's been found and reported upstream.

Validation and Vision

Does claude_toolkit validate my local Vision model?

No. The validation gate (Constitution §11.4.277) lives in llmctl: `llmctl validate`, `llmctl require-validated <profile>` and `LLMCTL_VALIDATE_STRICT=1`. When you launch `llmctl-vision`, the toolkit runs `llmctl switch vision`, passes your environment through, and aborts the launch if the gate refuses (exit 1 wrong answer, 2 undetermined, 75 pressure). It never launches against a profile the gate refused. This is covered by `scripts/tests/test_llmctl_vision_gate_wiring.sh`; the toolkit has no Vision known-answer suite of its own.

What does exit 2 / UNDETERMINED mean?

llmctl could not decide whether the model answers correctly (for example the engine was unreachable or host pressure refused the run). UNDETERMINED is never treated as a pass; under `LLMCTL_VALIDATE_STRICT=1` it stops the profile. Fix the cause (see `llmctl logs <profile>`), do not retry until it turns green.

Why does Vision QA (§11.4.278) matter here?

HelixQA's screen-reading oracle is a Vision model; it is only trustworthy while that model holds a fresh `require-validated` PASS. The toolkit supplies the `llmctl-vision` alias and keeps the gate intact; the QA itself is HelixQA's.

Is claude_toolkit part of the constitution?

It is a *hub engine*: consumed by reference at `constitution/submodules/claude_toolkit`, depth 1, never initialised recursively, dependencies resolved by `scripts/resolve_deps.sh` instead of nested copies (§11.4.28(C), §11.4.31). See the README's hub-engine section.